

# Multi-Agent Reinforcement Learning for No-Limit Texas Hold'em

Aléo Ibrahim Houmed & Maxence Adly  
Reinforcement Learning Group Project

## 1. Introduction & Motivation

**No-Limit Texas Hold'em (NLHE)** is one of the most challenging domains in AI due to its combination of:

- **Imperfect information** – players cannot see opponents' cards
- **Stochastic dynamics** – card dealing introduces randomness
- **Large action space** – bet sizing creates combinatorial complexity
- **Multi-agent competition** – opponents adapt their strategies
- **Sparse, delayed rewards** – outcomes only at hand conclusion

We train autonomous agents to play NLHE through **self-play reinforcement learning**, where a single shared policy improves by competing against copies of itself. This approach has proven effective in games like Go (AlphaGo/Zero) and poker (Pluribus, DeepStack).

### Key contributions:

- Custom multi-agent NLHE environment supporting 2–10 players
- PPO training with action masking for legal move enforcement
- Cash-game mode with persistent stacks and player elimination
- Interactive GUI and CLI tools for model evaluation

## 2. Game & Environment Design



### Environment specifications:

**Observation Space** (per player):

- Hole cards (one-hot, 52-dim)
- Board cards (multi-hot, 52-dim)
- Street indicator (one-hot, 4-dim)
- Position relative to dealer
- Normalised stacks, pot, bets
- Min/max raise amounts
- Per-street betting history

All values normalised to  $[0, 1]$ .

**Action Space** (6 discrete actions):

#### Idx Action

- |   |                         |
|---|-------------------------|
| 0 | Fold                    |
| 1 | Check / Call            |
| 2 | Raise Minimum           |
| 3 | Raise $\frac{1}{2}$ Pot |
| 4 | Raise Pot               |
| 5 | All-In                  |

**Reward:**  $r_i = \text{stack}_i^{\text{final}} - \text{stack}_i^{\text{initial}}$

Sparse, zero-sum at hand end.

## 3. Method: PPO with Self-Play

We use **Proximal Policy Optimization (PPO)** within the **Ray RLlib** framework, employing a single shared policy for all agents (*self-play*).

**PPO Objective** (clipped surrogate):

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[ \min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t) \right]$$

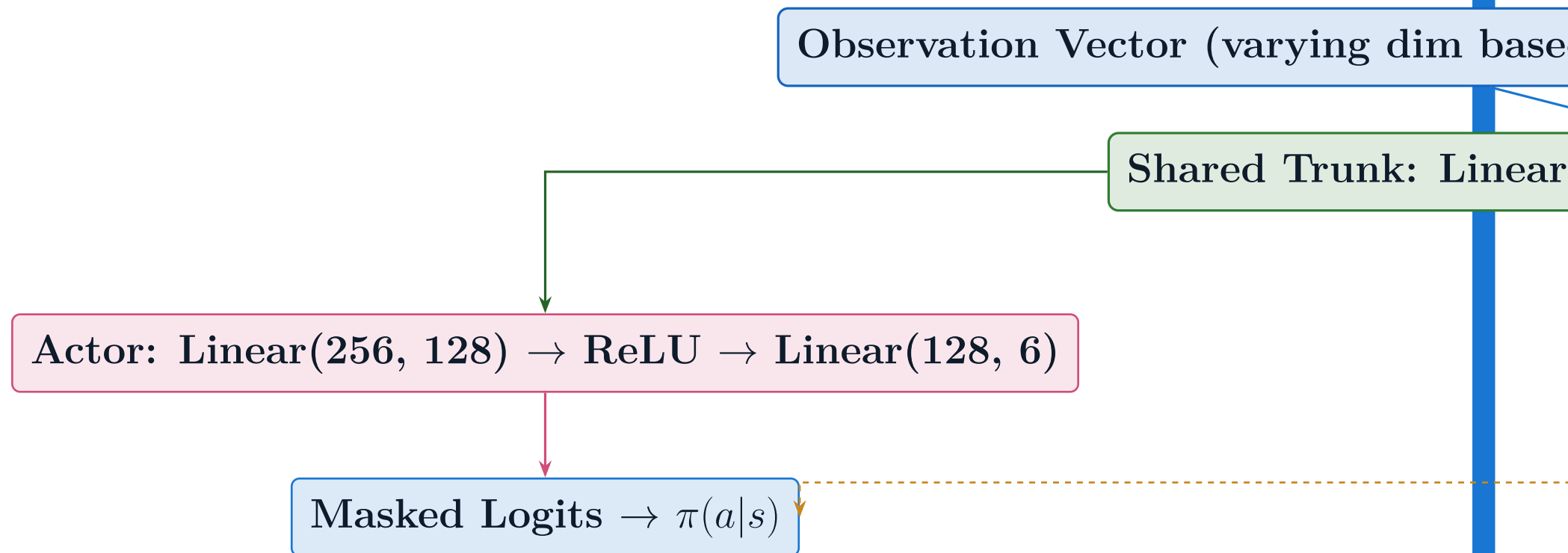
where  $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$  and  $\hat{A}_t$  is the GAE advantage estimate.

**Action Masking:** Illegal actions are suppressed by setting their logits to  $-10^{38}$  before the softmax, guaranteeing  $\pi(a|s) = 0$  for illegal moves:

$$\ell'_i = \begin{cases} \ell_i & \text{if action } i \text{ is legal} \\ -10^{38} & \text{otherwise} \end{cases}$$

| Hyperparameter    | Value              |
|-------------------|--------------------|
| Learning rate     | $3 \times 10^{-4}$ |
| Discount $\gamma$ | 1.0 (episodic)     |

## 4. Neural Network Architecture



The **PokerActionMaskRLModule** extends RLlib's **DefaultPPO TorchRLModule**:

- **Shared trunk:** Extracts features from the observation vector
- **Actor head:** Produces 6-dimensional logits, masked for legality
- **Critic head:** Outputs scalar baseline value  $V(s)$  for variance reduction
- **Total parameters:**  $\sim 200\text{K}$  (lightweight, fast inference)